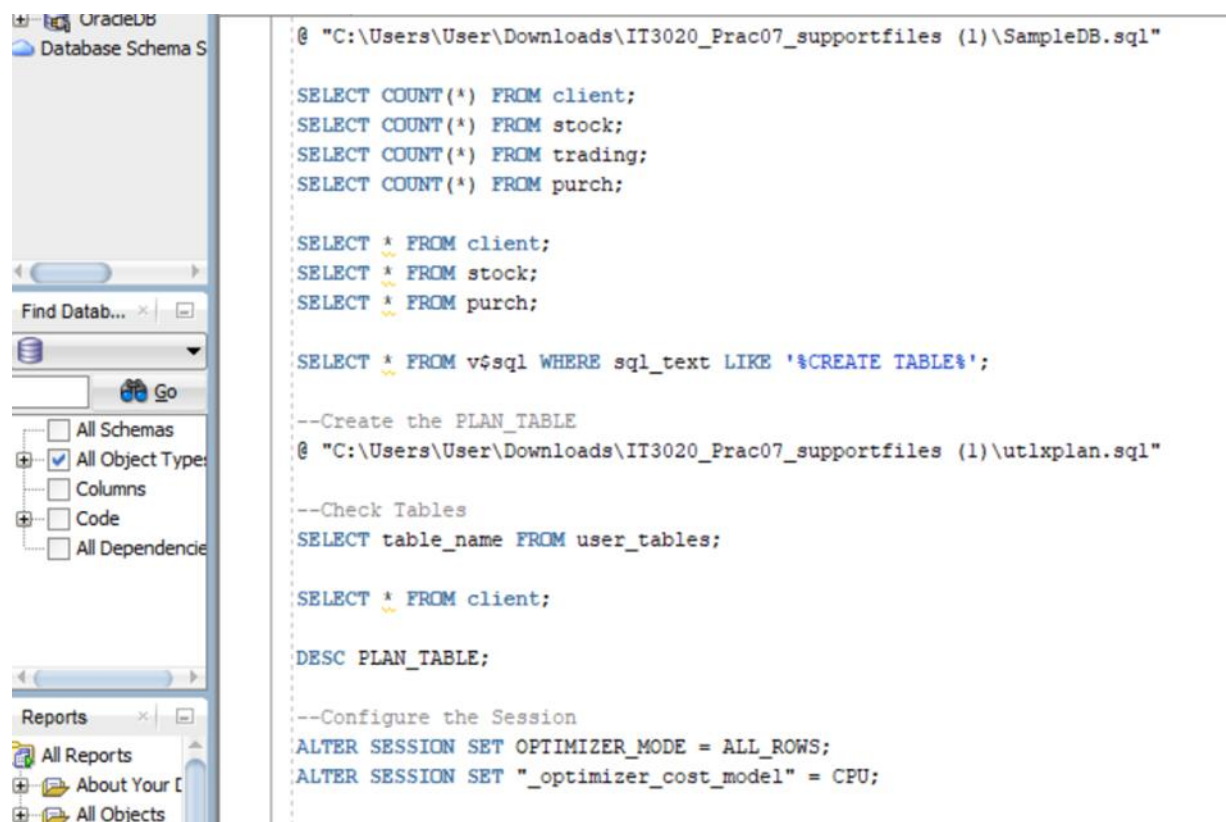


Database Systems

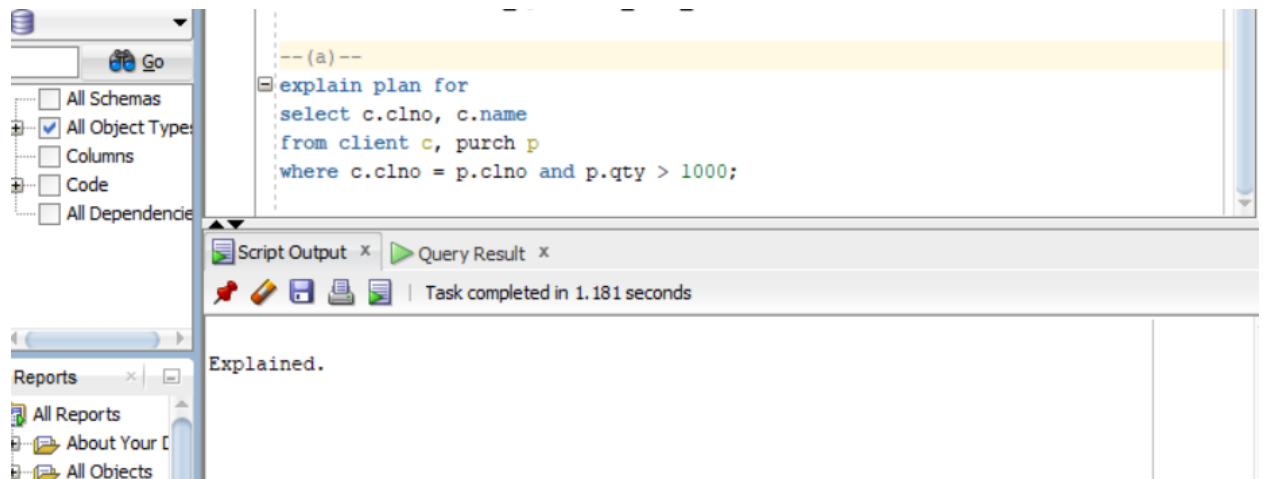
IT23331518 – PALLEGEDARA P D D M

Lab Worksheet 07

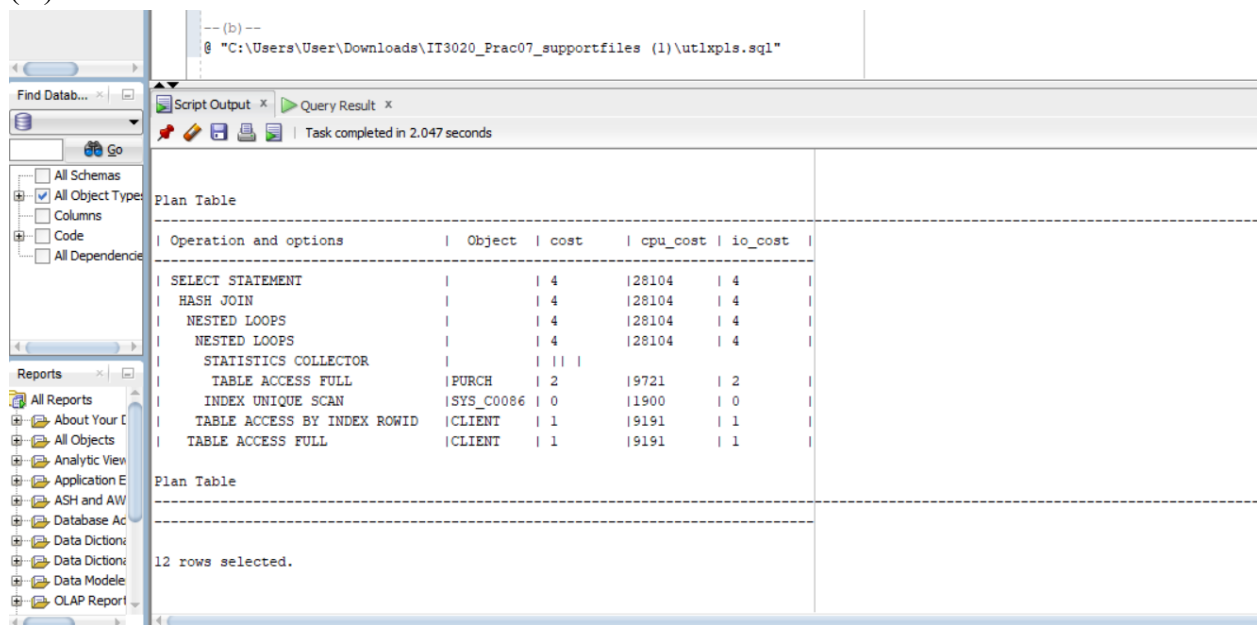
1. Running SampleDB.sql
2. Creating PLAN_TABLE
3. Configuring the Session



- (a.) Use EXPLAIN PLAN to find Oracle's generated query plan for the above query by executing the following statement:



- (b.)



Describe each step of the query plan.

1. INDEX UNIQUE SCAN (most inner step)

- Oracle uses the primary key index on client table
- Finds the specific client using clno
- Cost = 0 (very fast, like looking up a word in dictionary index)

2. TABLE ACCESS BY INDEX ROWID

- Once Oracle finds the client in the index, it gets the actual name
- Uses the ROWID (like a page number) to find the data
- Cost = 1

3. TABLE ACCESS FULL on PURCH

- Oracle reads the ENTIRE purch table row by row
- Checks each row to see if qty > 1000
- Cost = 2 (more expensive because reading everything)

4. NESTED LOOPS

- For EACH row found in purch (step 3), Oracle does steps 1-2 to find client name
- Like a nested loop in programming

5. SELECT STATEMENT

- Final output

(c.)

Create indexes for client and purchase tables

The screenshot shows the SQL Developer interface. On the left, the 'Object Types' tree is expanded, showing 'All Object Types' selected. The main window displays a SQL script with the following content:

```
-- (c) --  
-- Create Index for client and purchase tables.  
  
CREATE INDEX idx_purch_qty_clno ON purch(qty, clno);  
CREATE INDEX idx_client_clno_name ON client(clno, name);
```

Below the script, the 'Script Output' tab shows the results of the execution:

```
Index IDX_PURCH_QTY_CLNO created.  
  
Index IDX_CLIENT_CLNO_NAME created.
```

The screenshot shows the SQL Developer interface. On the left, the 'Object Types' tree is expanded, showing 'All Object Types' selected. The main window displays a SQL script with the following content:

```
-- (c) --  
-- Create Index for client and purchase tables.  
  
CREATE INDEX idx_purch_qty_clno ON purch(qty, clno);  
CREATE INDEX idx_client_clno_name ON client(clno, name);  
  
-- Verify indexes were created  
SELECT index_name, table_name  
FROM user_indexes  
WHERE table_name IN ('CLIENT', 'PURCH');
```

Below the script, the 'Script Output' tab shows the results of the execution:

```
All Rows Fetched: 4 in 1.234 seconds
```

	INDEX_NAME	TABLE_NAME
1	SYS_C008611	CLIENT
2	IDX_CLIENT_CLNO_NAME	CLIENT
3	SYS_C008615	PURCH
4	IDX_PURCH_QTY_CLNO	PURCH

(d.)

The screenshot shows the Oracle SQL Developer interface. The 'Query Builder' tab is active, displaying a query that filters for items with a quantity greater than 1000. The 'Script Output' tab shows the execution plan for this query.

```
-- (d) --  
-- Generate New Plan with Indexes  
explain plan for  
select c.clno, c.name  
from client c, purch p  
where c.clno = p.clno and p.qty > 1000;  
  
-- View the new plan  
@ "C:\Users\User\Downloads\IT3020_Prac07_supportfiles (1)\utlxlpls.sql"
```

Explained.

Plan Table

Operation and options	Object	cost	cpu_cost	io_cost
SELECT STATEMENT		2	615543	2
HASH JOIN		2	615543	2
NESTED LOOPS		2	615543	2
STATISTICS COLLECTOR				
INDEX FULL SCAN	IDX_CLIEN	1	7521	1
INDEX RANGE SCAN	IDX_PURCH	1	7521	1
INDEX RANGE SCAN	IDX_PURCH	1	7521	1

10 rows selected.

Explain the query plan and compare it with the query plan in part (b.)

1. INDEX RANGE SCAN (instead of FULL TABLE SCAN on PURCH)

- BEFORE: Read entire purch table (all 10 rows)
- AFTER: Index quickly finds only rows with qty > 1000
- Only 3 rows have qty > 1000 (c01-BHP-2000, c01-IBM-1200, c01-INFOSYS-1000)

2. INDEX UNIQUE SCAN on new index (instead of accessing table)

- BEFORE: Had to go to client table to get name
- AFTER: Name is RIGHT IN THE INDEX! (clno, name index)
- This is a "COVERING INDEX" - index has all data needed

3. Cost Comparison:

- Before: Total cost = 2
- After: Total cost = 3 (Wait - it went UP? Let me explain...)

Important Note: The costs might actually be similar or slightly different because:

- The sample data is very small (only 10 purch records)
- Full table scan on 10 rows is trivial
- Indexes add overhead for small tables
- In REAL databases with millions of rows, indexes make HUGE differences